COURSE SYLLABUS

วิชา	**2110473 Fault Tolerant Computing**
จำนวนหน่วยกิต (Course Credit)	3 หน่วยกิต
ชื่อผู้สอน	ผศ.ดร.อาทิตย์ ทองทักษ์ (Rm.no.18—04)
	Arthit.t@chula.ac.th

วัตถุประสงค์ทั่วไป เพื่อให้ผู้เรียนได้

1. เข้าใจทฤษฎี ความรู้พื้นฐานและมีโอกาสติดตามเทคโนโลยีที่เกี่ยวข้องกับ Fault Tolerant Computing
2. เข้าใจถึงขั้นตอนวิธีการในการวิเคราะห์และการออกแบบระบบที่ดำเนินการแบบ Fault Tolerant Computing
3. มีประสบการณ์ในการใช้เครื่องมือการออกแบบระบบที่ดำเนินการแบบ Fault Tolerant Computing

เนื้อหารายวิชา (อาจมีการเปลี่ยนแปลงได้ตามความเหมาะสม)

1. Dependable Computing: Terminology
2. Fault Tolerant Computing: Basic Concept
3. Redundancy Technique: Hardware, Software, Information, Time
4. Error Detection & Error Correction
5. Modeling & Evaluation: Fault Modeling & Reliability
6. Testing Digital Circuits
7. Design for Test

การวัดผลเรียน

1. Midterm Test 30%
2. Final Assignment 20%
3. Homework 50%

หนังสืออ่านประกอบ

1. Several Sources

Contact

- **MyCourseVille:**

  2110473(2026/1) FAULT TOLERANT COMPUTING [Section21&51]

  passwd: 2110473
- **Discord:**

  https://discord.gg/q7uGHkGSw
- **Office Hours:**

  room: #18-04, Bld: Eng.no.4, Fri 9.30-11.00
- **Others:**

  arthit.t@chula.ac.th

  FB messenger: Arthit Thongtak

Weekly Plan (tentative)

Date 1	Week	Chapter	Class	Assignment
2026-08-03	1	Adminstrative issues	Eng3-421	
2026-08-10	2	FTC01 Dependable Computing	TBA	1
2026-08-17	3	FTC02 Fault Tolerant Systems	TBA	2
2026-08-24	4	FTC02plus Checkpointing and Recovery	TBA	3
2026-08-31	5	FTC03 Coding Theory	TBA	4
2026-09-07	6	FTC04 Self Checking	TBA	5
2026-09-14	7	FTC05 Fault Masking and Recovery	TBA	6
2026-09-21	8	**Midterm week**		
2026-09-28	9	Review	ZOOM	
2026-10-05	10	Review	ZOOM	
2026-10-12	11	**MIDTERM TEST**	TBA	
2026-10-19	12	TDC01 Overview of Testing Digital Circuits	ZOOM	7
2026-10-26	13	TDC02 Delay Fault Testing	ZOOM	8
2026-11-02	14	TDC03 Digital Circuit Testing Issues	ZOOM	9
2026-11-09	15	TDC04 Asynchronous Circuit Testing	ZOOM	10
2026-11-16	16	Review	ZOOM	**FINAL ASSIGNMENT**
2026-11-23		**Final week**		*(as of Aug.03, 2025)*

Dependable Computing*

Concepts, Limits, Challenges

2110473 Fault Tolerant Computing

* Jean-Claude Laprie, Invited Paper to FTCS-25, CAL USA, Jun27-30, 1995, Special Issue pp.42-54

Introduction

- Increasingly Depended on Computing Systems
- However, This Dependency felt upon Failures
  - Telephone Outage in USA, 1990
  - Credit Card Denial of Authorization in FRA, 1993
  - London Ambulance Service Failure, 1992 etc
- Computing finally resulted in both Benefits & Costs



- “What are the Limits of Computing Systems Dependability?”
- “What are the Challenges?”

Dependability Concept

- Definitions of Dependability
  - [classic ‘60] The Probability that a System will operate when needed
  - [review ‘95] Integrative Concept
  - The Property of a Computer System such that
    - Delivery of Service that can justifiably be Trusted
      - Availability, reliability, safety, confidentiality, integrity, maintainability, that are then attributes of dependability
    - Avoidance of Service failures that are unacceptably frequent or severe
      - Frequency & severity, criterion, dependability failure

Basic Concepts and Taxonomy of Dependable and Secure Computing; Avizienis, Laprie, Randell, and Landwehr; IEEE Trans.on Dependable and Secure Computing, Vol.1, No.1

Avoidance of Service Failures

- Failure Frequency & Severity

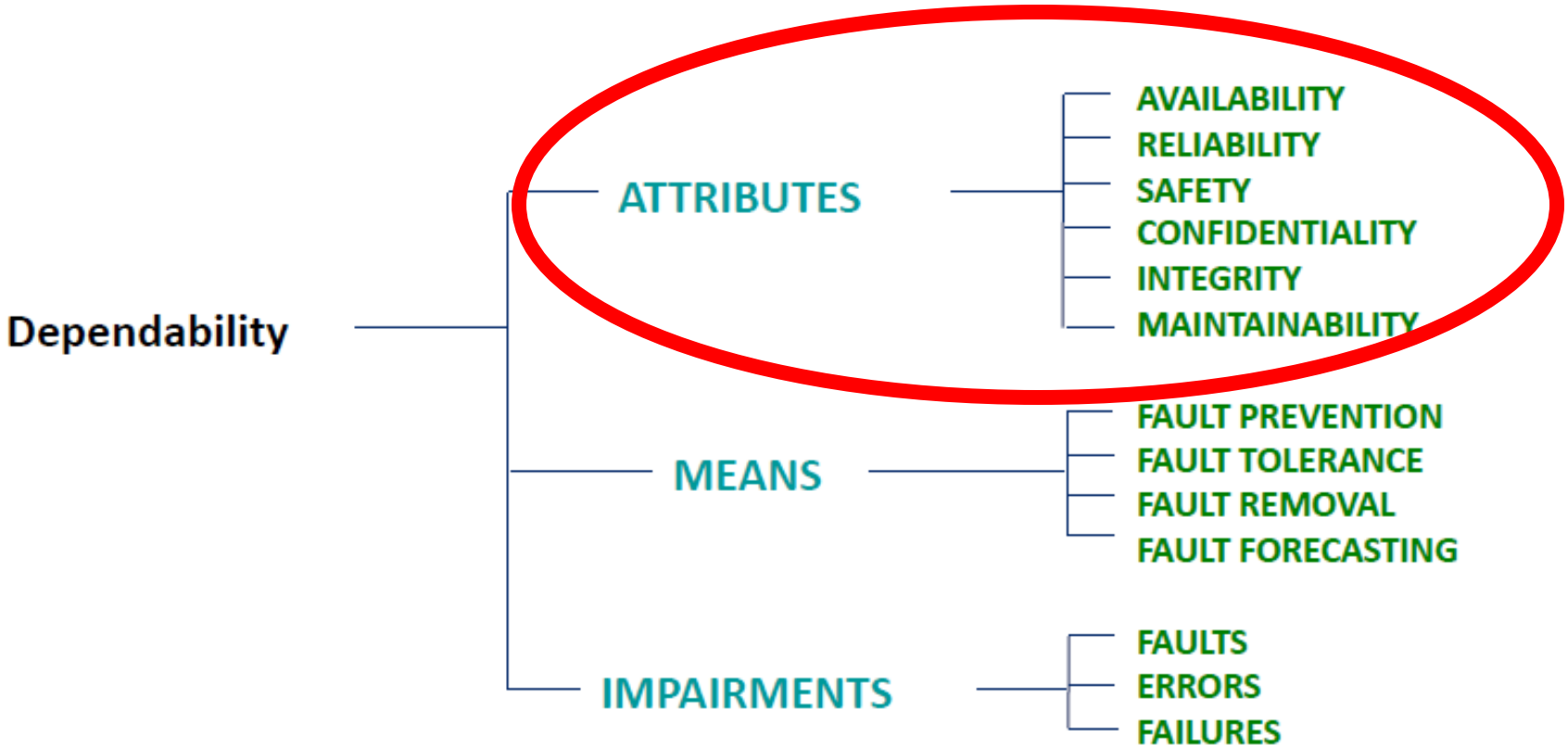
  $\Rightarrow$ Risk Assessment & Management
- Criterion for System Dependable even of Service Failures
- Unacceptably Frequent or Severe Service Failures

  $\Rightarrow$ Dependability Failure, Connected to Development Process Failure

Classifications of Dependability

- Properties which are expected from the Systems:
  - Dependability **Attributes**
- Methods/Techniques to provide the Abilities:
  - Dependability **Means**
- Causes or Resulting from Un-Dependability:
  - Dependability Threats/**Impairments**

Dependability: Attributes, Means and Impairments



Dependability Attributes

- Property-based Attributes
  - Availability, Reliability, Safety, Confidentiality, Integrity, Maintainability
- Event-based Attributes
  - Robustness, Survivability, Resilience
- Attributes based on meta-information
  - Accountability, Authenticity, Non-Repudiability

Property-based Attributes

- Availability
  - The Readiness for Usage
- Reliability
  - The Continuity of Service
- Safety
  - The Non-occurrence of Catastrophic on the environment
- Confidentiality
  - The Non-occurrence of Unauthorized Disclosure of information
- Integrity
  - The Non-occurrence of Improper Alterations of Information
- Maintainability
  - The ability to Undergo Repairs & Evolutions

** Security: Availability & Integrity wt respect to Authorized Actions, together wt Confidentiality

Examples of Attributes

Type of system	Availability	Reliability	Safety	Integrity	Maintainability
Aircraft systems	X	X	X	X	X
Nuclear reactor	X	X	X		X
Internet News page	X	X		X	X
Smart Phone	X		X	X	X
Pace Maker	X	X	X	X	
Automobile System		X	X	X	X

Historical Evolution

- 40's: ENIAC
  - 18K vacuum tubes → failed ~ every 7 mns
  - 18K multiplies/minute → 7 mns ~ one program execution

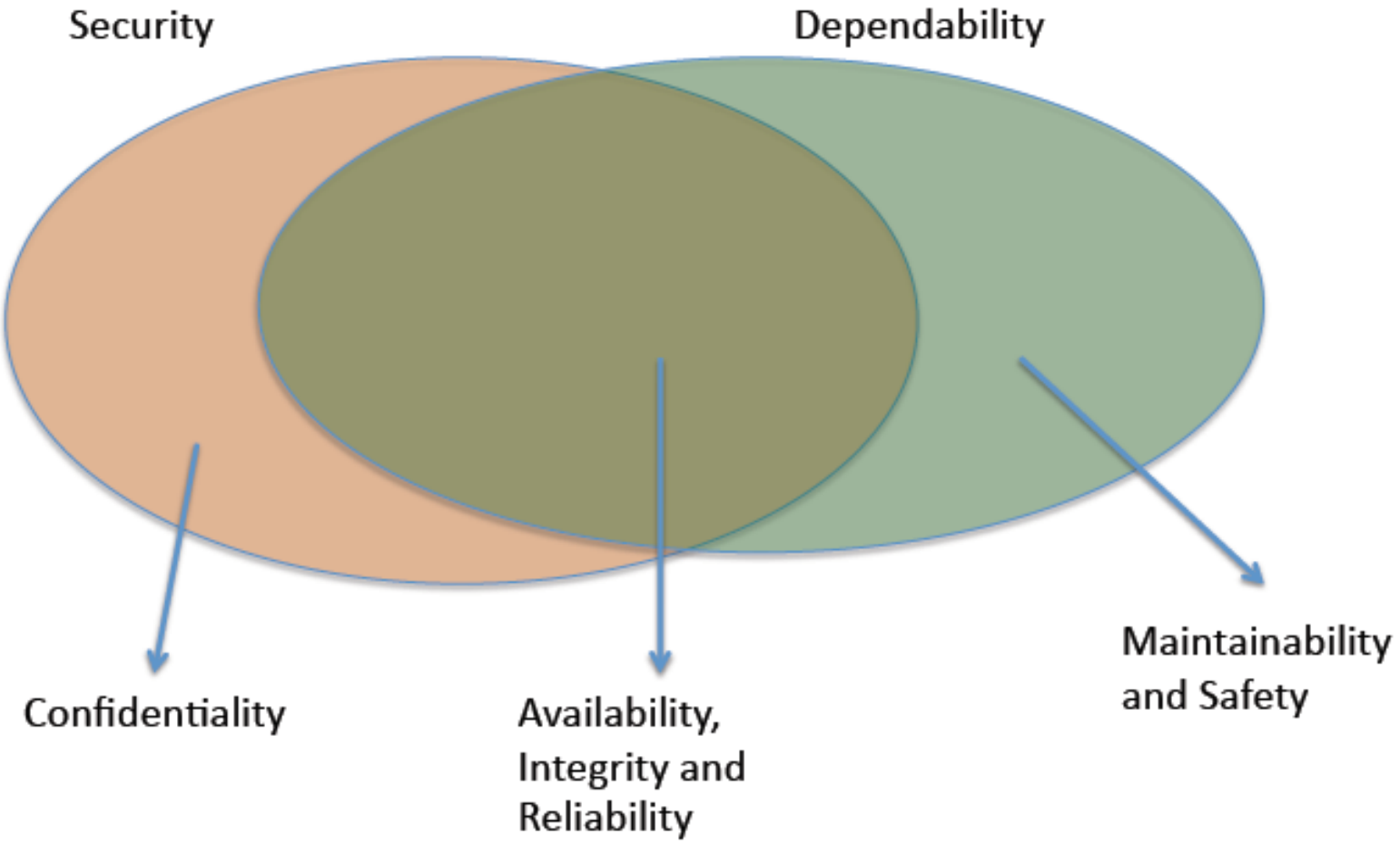
  Need RELIABILITY
- 60's: Interactive systems

  + AVAILABILITY
- 70's: F-8 Crusader, MD-11, Patriot missile defense

  + SAFETY
- 90's-today: Internet, E-commerce, Grid/Web services

  + SECURITY

Dependability and Security



Availability

- a ratio of the expected value of the uptime of a system to the aggregate of the expected values of up and down time

$$A = \frac{E[\text{Uptime}]}{E[\text{Uptime}] + E[\text{Downtime}]}$$

Example of Availability

- If we are using equipment which has a mean time to failure (MTTF) of 81.5 years and mean time to repair (MTTR) of 1 hour:
- MTTF in hours = 81.5*365*24=713940 (This is a reliability parameter and often has a high level of uncertainty!)
- Inherent Availability (Ai) = MTTF/(MTTF+MTTR) = 713940/713941 =99.999859%
- Inherent Unavailability = 0.000141%
- Outage due to equipment in hours per year = 1/rate = 1/MTTF = 0.01235 hours per year

Nines of Availability

# 9's	%	Downtime / year	Systems
2	99%	~5000 minutes	General web site
3	99.9%	~500 minutes	Amazon.com
4	99.99%	~50 minutes	Enterprise server
5	99.999%	~5 minutes	Telephone System
6	99.9999%	~30 seconds	Phone switches

Reliability

- The ability of an item to perform a required function under stated conditions up until specified time

$$m = \frac{1}{\lambda}$$

$$R_{(t)} = e^{-\lambda t} = e^{-(t/m)}$$

$$m = \frac{t}{\log_{(n)}\left(\frac{1}{R_{(t)}}\right)}$$

Where $R_{(t)}$ = reliability
e = exponential (2.178)
λ = failure rate
m = mtbf
t = time

Reliability vs Availability

- They are not the same …
- Example:

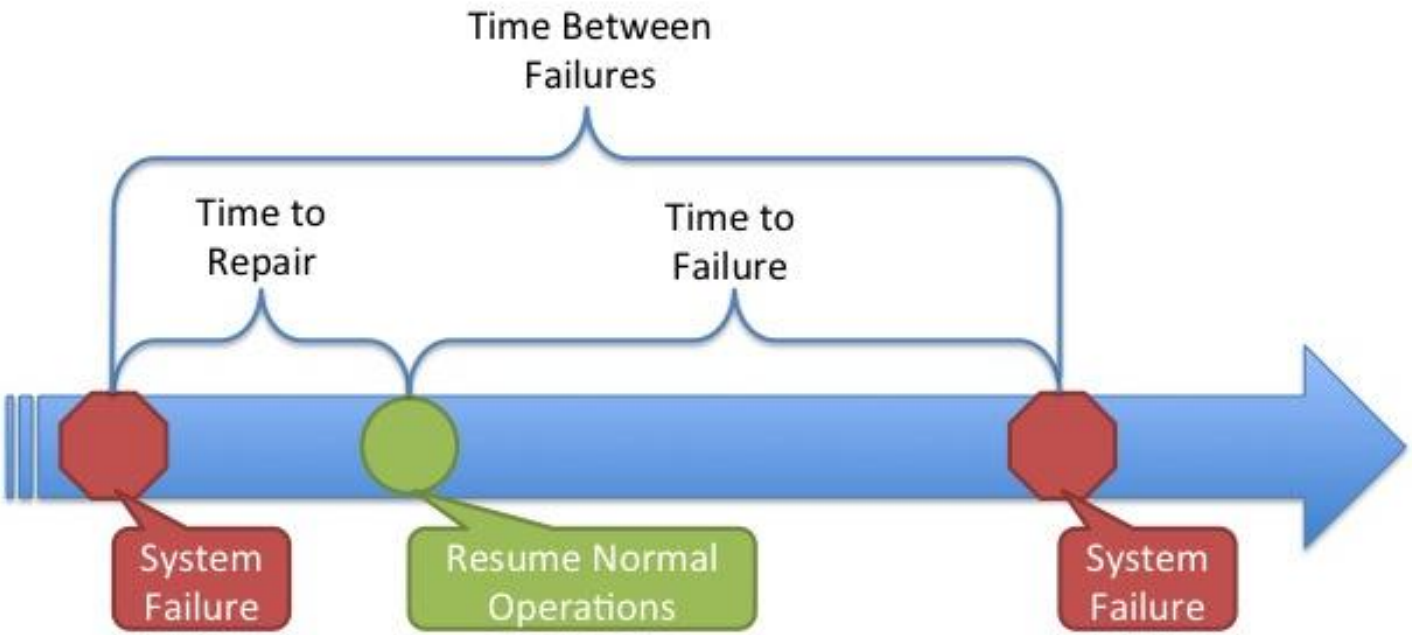
A system that fails, on average, once per hour but which restarts automatically in ten mil.secs.



Non-reliable

but highly available

Failure Metrics



Design Tradeoffs

Design Tradeoffs

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

MTTF = Mean Time To Failure

MTTR = Mean Time To Repair

- How to make availability approach 100%?

  MTTF → infinity (high reliability)
  MTTR → zero (fast recovery)

- Need to define maximum repair time
- Need to define how Availability is measured

Safety

- ***"The most important things cannot be measured."*** **- W. Edwards Deming**
- "Not everything that counts can be counted, and not everything that can be counted counts." – Albert Einstein
- **Lost time injury/incident frequency rates (LTIFR)**

$$\frac{\textit{(Number of lost time injuries in accounting period)}}{\textit{(Total hours worked in accounting period)}}$$

Safety (cont.)

Absence of catastrophic consequences on the users or the environment

- Are commercial aircraft “safe”?
- They crash occasionally
- How many crashes are too many?
- Are cars “safe”? They crash quite a lot

40K deaths/yr; 800/week = 2 fully loaded Boeing 747/week

Risk

- *Risk* is defined to be expected loss per unit time:

$$\text{Risk} = \sum \text{pr}(\text{accident}_i) \times \text{cost}(\text{accident}_i)$$

- *Safety is defined to be an acceptable level of risk*:

$$\text{Risk} < \text{pre-defined_delta}$$

Reliability vs Availability vs Safety

- They are not the same …
- Example:

  A system that is turned off …

  not reliable, not available, but safe

CIA Triad

- A model designed to guide policies for information security within an organization



- **Confidentiality** is a set of rules that limits access to information
- **Integrity** is the assurance that the information is trustworthy and accurate
- **Availability** is a guarantee of reliable access to the information by authorized people

Confidentiality

- Prevent sensitive information from reaching the wrong people, while making sure that the right people can in fact get it
- Confidentiality → Information; Privacy → People
- Confidentiality methods (Social Engineering):
  - *Data encryption, IDs and passwords*
  - *Two-factor authentication*
  - *Biometric verification*
  - *Security tokens, Key fobs or soft tokens*

Integrity

- Integrity involves maintaining the consistency, accuracy, and trustworthiness of data over its entire life cycle
- Data Integrity → Overall aspects

  Data Security → Unauthorized parties
- Physical Integrity:
  - *Redundant hardware, UPS*
  - *RAID, Error-correcting codes*
  - *Cryptographic hash function, Watchdog timer*
- Logical Integrity:
  - *File permission, Access control, Version control*
  - *Referential integrity and entity integrity in a relational database*
  - *Software bugs, Design flaws, Human errors*

Data Quality VS Data Integrity

- **Data Quality:** *refer to whether data is useful*
  - **Completeness –** The data present is a large percentage of the total amount of data needed.
  - **Uniqueness –** Unique data sets are free of redundant or extraneous entries.
  - **Validity –** Data conforms to the syntax and structure defined by the business requirements.
  - **Timeliness –** Data is up to date or relevant to the timelines referenced by the business requirements.
  - **Accuracy –** Data accurately describes the real-world context it refers to.
  - **Consistency –** Data is consistent throughout the dataset.
- **Data Integrity:** *refer to whether data is trustworthy*
  - **Physical Integrity –** Utilizing data is difficult when there are problems either storing or retrieving it. It raises questions about the data's completeness, accuracy, and validity. The physical integrity of data hardware can become compromised due to age, malfunction, maintenance problems, natural disasters, or power outages.
  - **Logical Integrity –** Data becomes illogical when it is incorrect or irrational in some way. Data has to "make sense" for its context. Otherwise, it distorts the perspective of anyone basing decisions off the data. Logical problems can happen because of design flaws, human errors, or software bugs.

RAM/RMA

- Reliability + Availability + Maintainability
- System design attributes that impacts on the sustainment or total Life Cycle Costs (LCC) of a developed system
- Test Data under RAM concept includes:
  - *Software Functionality*
  - *Operational Quality*
  - *Known Remaining Defects (defect density)*
  - *Testing Scope and Stability*
  - *Reliability*

Classification of Maintenance

- Breakdown Maintenance/ Repair
  - Perform system maintenance/repair after breakdown
- Time-based (Preventive) Maintenance
  - Perform system maintenance periodically
- Condition-based (Preventive) Maintenance
  - Perform system maintenance upon the specified conditions
- Predictive Maintenance
  - Predict the life of system components in order to perform system maintenance before breakdown

Event-based Attributes

- Robustness
  - Persistence of Dependability wrt External Faults
- Survivability
  - Persistence of Dependability in the presence of Active Fault(s)
- Resilience
  - Persistence of Dependability when facing Functional, Environmental, or Technological Changes

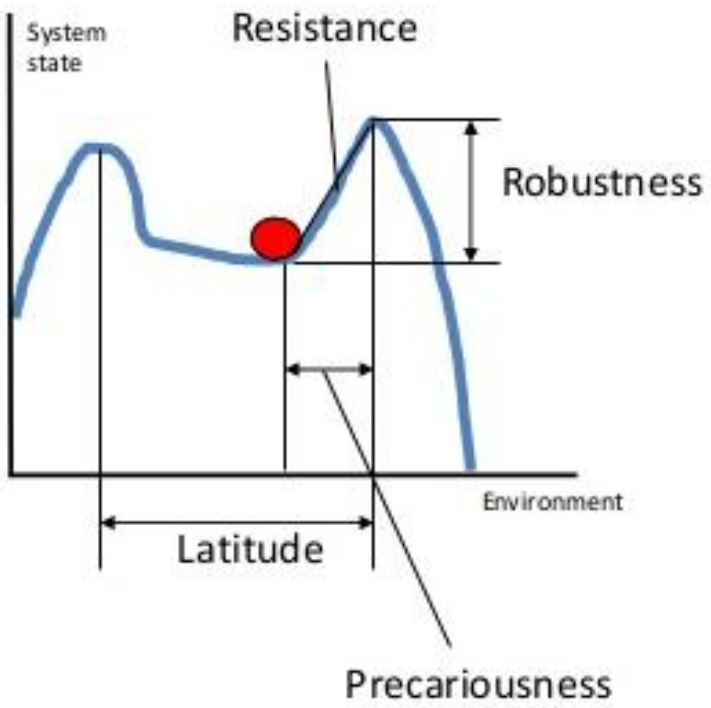
Robustness & Stability

- Commonalities:
  - Defined for specified features of a given system, with *specified perturbations* being applied to the system
  - Both are concerned with the *persistence*
- Differences:
  - Robustness addresses behavior in a *more varied class* of systems, perturbations, & features of persistence
  - Robustness *lies outside the purview of stability* including organizational architecture, interplay btw organization & dynamics, relation to evolvability in the past and future, ability of the system to switch among multiple functionalities, & so on



Robustness beyond Stability

Characteristics of Resilient Systems



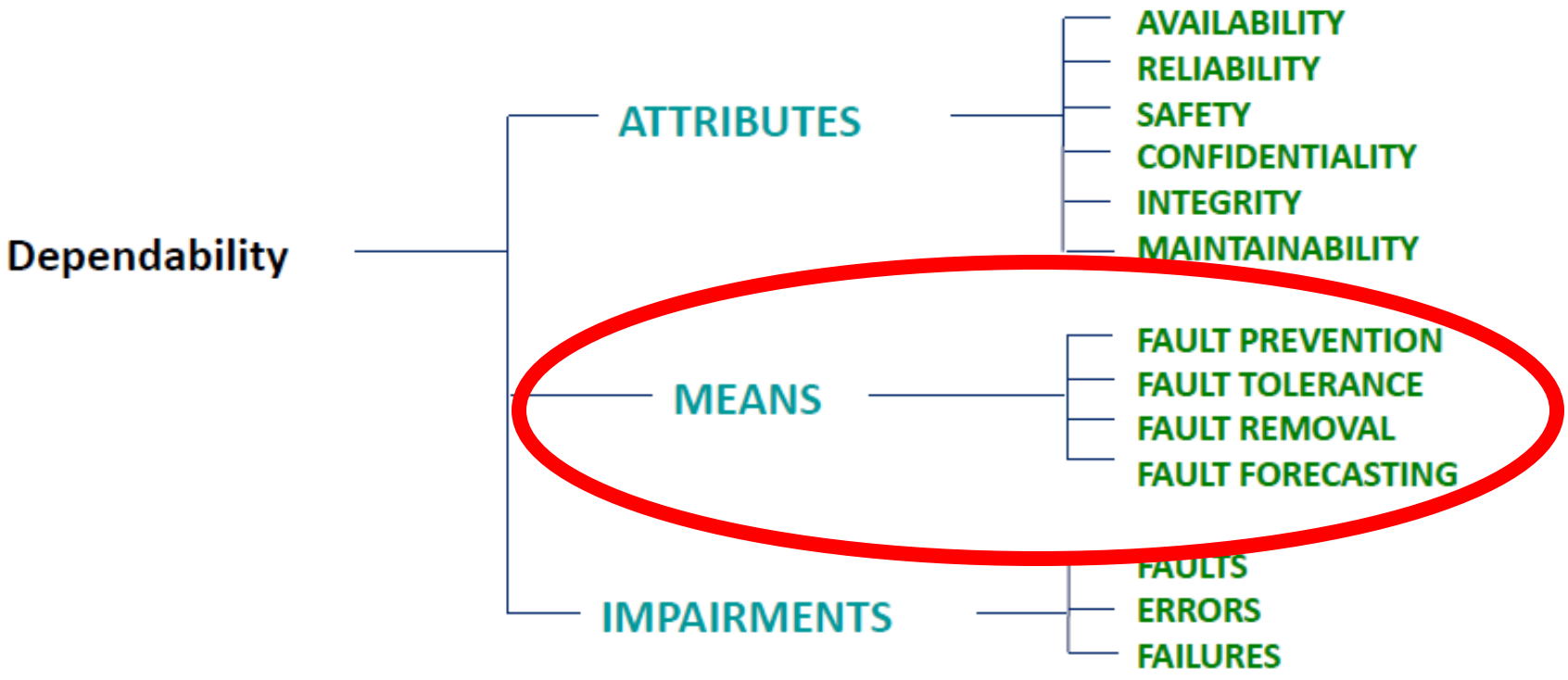
- Latitude: the maximum amount a system can be changed before losing its ability to recover (before crossing a threshold which, if breached, makes recovery difficult or impossible)
- Resistance: the ease or difficulty of changing the system; how "resistant" it is to being changed
- Precariousness: how close the current state of the system is to a limit or "threshold"
- *Panarchy: the degree to which a certain hierarchical level of an ecosystem is influenced by other levels*

Closely linked to Adaptive Capacity

Attributes based on meta-information

- Accountability
  - Availability and Integrity of the Person who performed an Operation
- Authenticity
  - Integrity of Message Content & Origin, & Possibly some other info, such as Time of Emission
- Non-Repudiability
  - Availability & Integrity of the Identity of the Sender of a Message, or of the Receiver

Dependability:
Attributes, Means and Impairments



Exercise

Example Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 SCSI controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 SCSI cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Answer

Example Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 SCSI controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 SCSI cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Answer The sum of the failure rates is

$$\text{Failure rate}_{\text{system}} = 10 \times \frac{1}{1{,}000{,}000} + \frac{1}{500{,}000} + \frac{1}{200{,}000} + \frac{1}{200{,}000} + \frac{1}{1{,}000{,}000}$$

$$= \frac{10 + 2 + 5 + 5 + 1}{1{,}000{,}000 \text{ hours}} = \frac{23}{1{,}000{,}000} = \frac{23{,}000}{1{,}000{,}000{,}000 \text{ hours}}$$

or 23,000 FIT. The MTTF for the system is just the inverse of the failure rate:

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1{,}000{,}000{,}000 \text{ hours}}{23{,}000} = 43{,}500 \text{ hours}$$

or just under 5 years.

Dependability Means

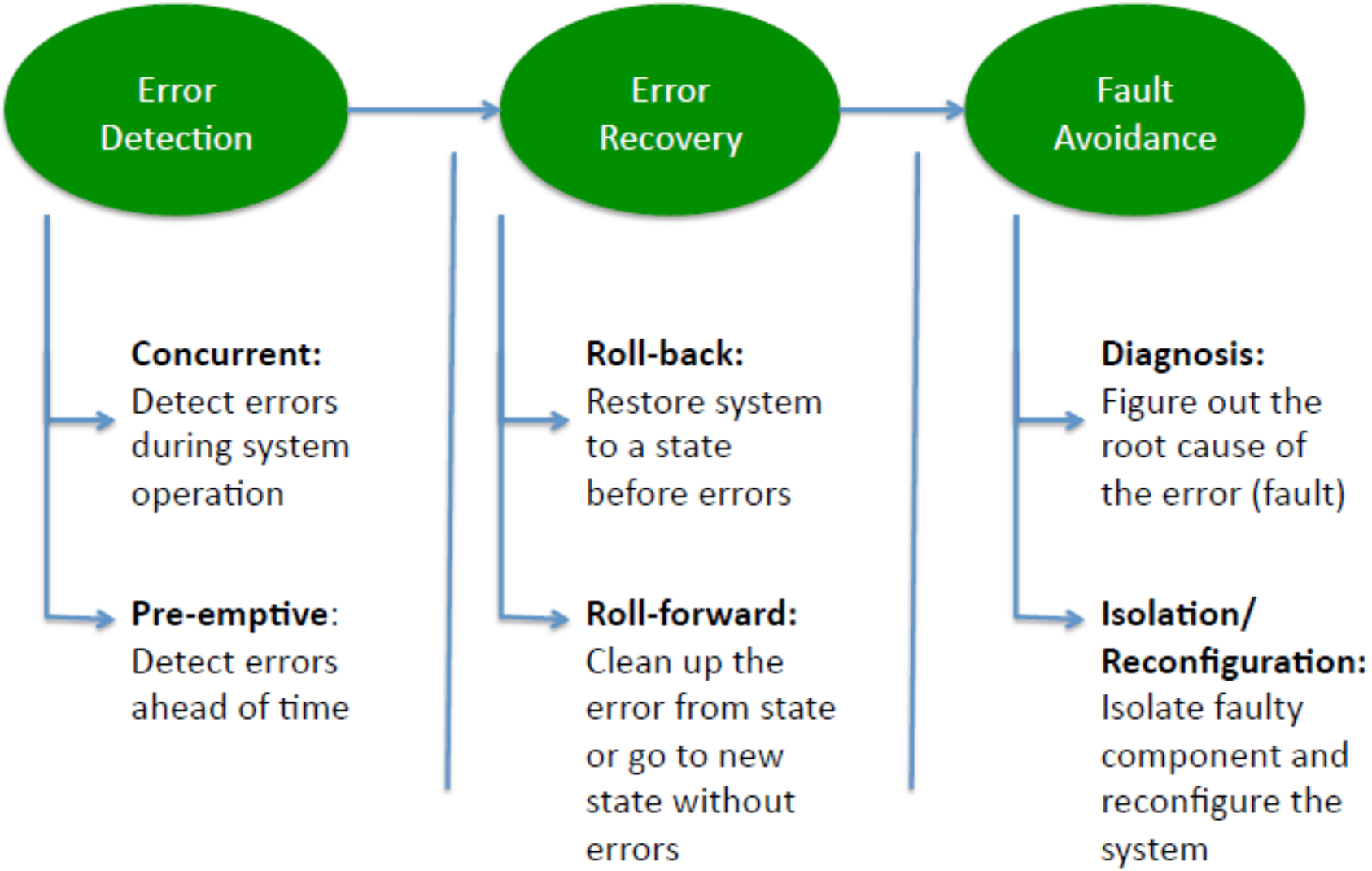
- Fault Prevention
  - To Prevent Fault Occurrence or Introduction
- Fault Tolerance
  - To Ensure a service upto Fulfilling the System's Function in the Presence of Faults
- Fault Removal
  - To Reduce the Presence of Faults
- Fault Forecasting
  - To Estimate the Present Number, Future Incidence, & the Consequences of Faults

Examples

Consider a space mission that has to be deployed for long periods of time (say a few years)

- **Fault Prevention**: Use of reliable coding practices, (e.g., defensive programming) and safe languages
- **Fault Tolerance**: Provide ability to detect and repair failed components during missions
- **Fault Removal**: Remove the faults at runtime or record the commonly observed faults for later removal
- **Fault Forecasting**: Predict how likely a future mission is to succeed based on the lessons learned

Fault Tolerance Techniques



Fault Tolerance

The ability to provide continued correct operation despite the presence of faults

- Encompasses a broad rage of techniques ranging from low-level devices to application software
- Important to ensure that the service behaves as expected provided fault belongs to fault-model
- There is no such thing as perfect fault-tolerance. Every fault-tolerance technique has a coverage and a fault-model over which it is evaluated

Error Detection

- **Concurrent detection** during normal operation
  - Watchdog timer
  - Software assertions
  - Process pairs
- **Preemptive detection** preempts the failure
  - Spare checking
  - Memory scrubbing
  - Software rejuvenation

Error Recovery

- **Rollback**

  Restore the state of the system to one before the fault. Useful for transient and intermittent faults

- **Rollforward**

  Correct the fault and allow system to make forward progress, if possible

- **Compensation**

  Leverage natural redundancy of the system to mask the error

Fault Avoidance

- Diagnosis
  - Identify the root cause of the fault (location, type)
- Isolation
  - Physically/Logically exclude faulty component
- Reconfiguration
  - Switch in non-faulty components and remap the tasks to the non-faulty components
- Reintialization
  - Record the new system state and update the external entities that interface with the system (if necessary)

Example: Hardware Fault Tolerance

- Multicore system experiences a fault on one of the cores. Error is **detected** through a concurrent check. What are the possible recovery actions that can be taken?

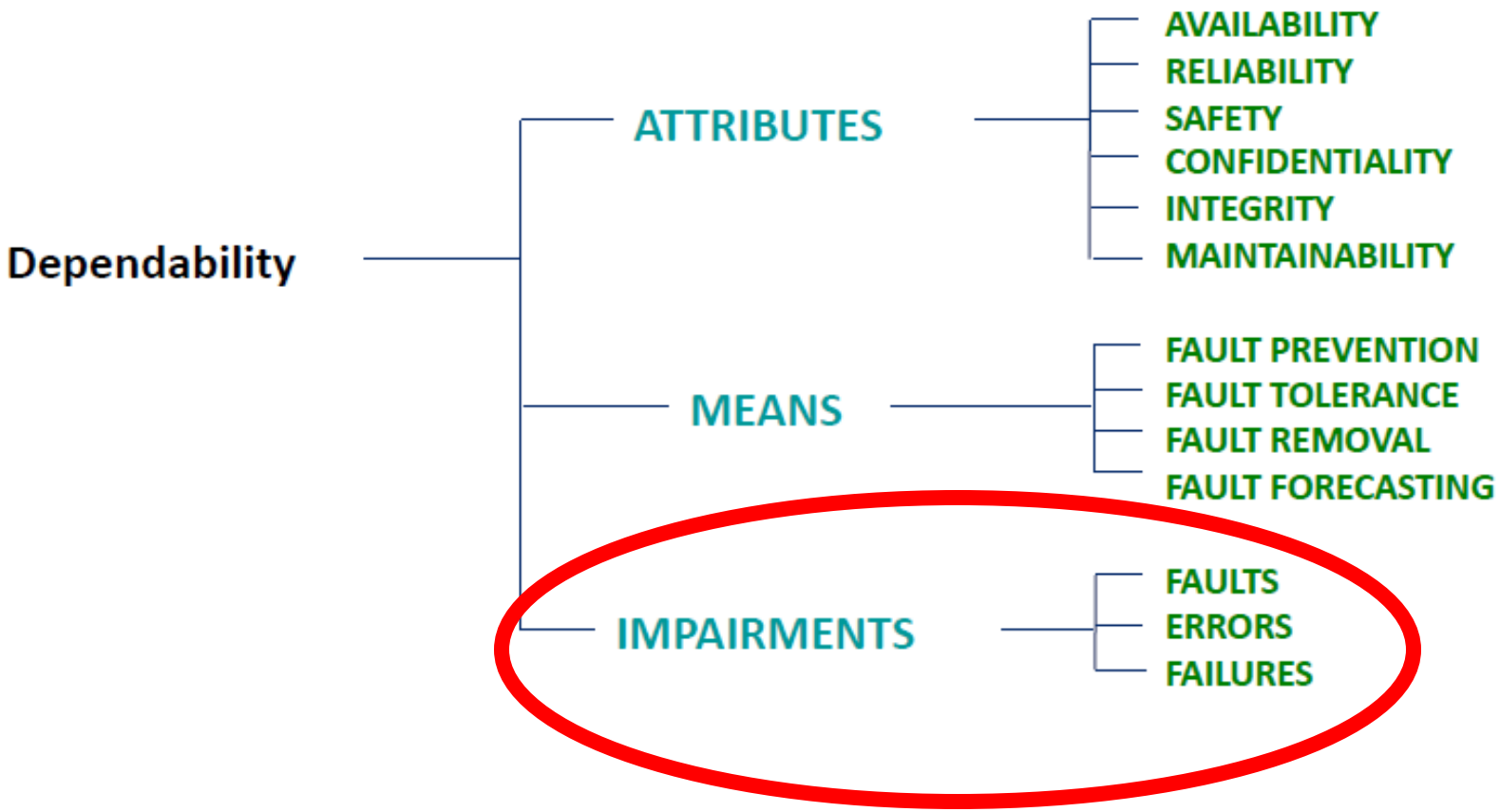
Example: Software Fault Tolerance

- Two identical copies of a software program are run concurrently to check each other. If the output of one differs from the output of the other, what recovery actions can be taken?

Coverage

- Every fault-tolerance technique has associated coverage, i.e., the probability that it detects/ corrects a fault, given that a fault has occurred
  - Coverage can be reduced by defects in the fault-tolerance mechanism's implementation ("Who will check the checker?")
  - Coverage can be reduced because of common mode failures or because assumptions made during the FTM's design are violated in practice

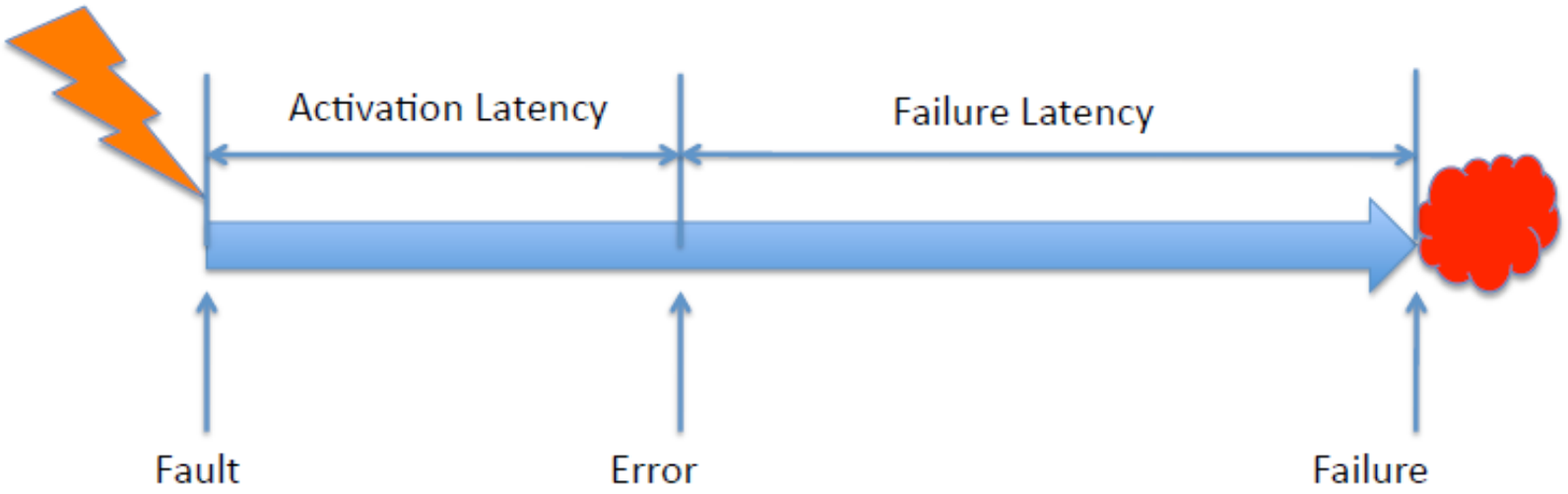
Dependability: Attributes, Means and Impairments



Dependability Threats/Impairments

- Faults
  - The adjudged or hypothesized Cause of an error
- Errors
  - Part of the System state that may lead to a Subsequent Failure
- Failures
  - Delivered Service Deviates from the Correct Service

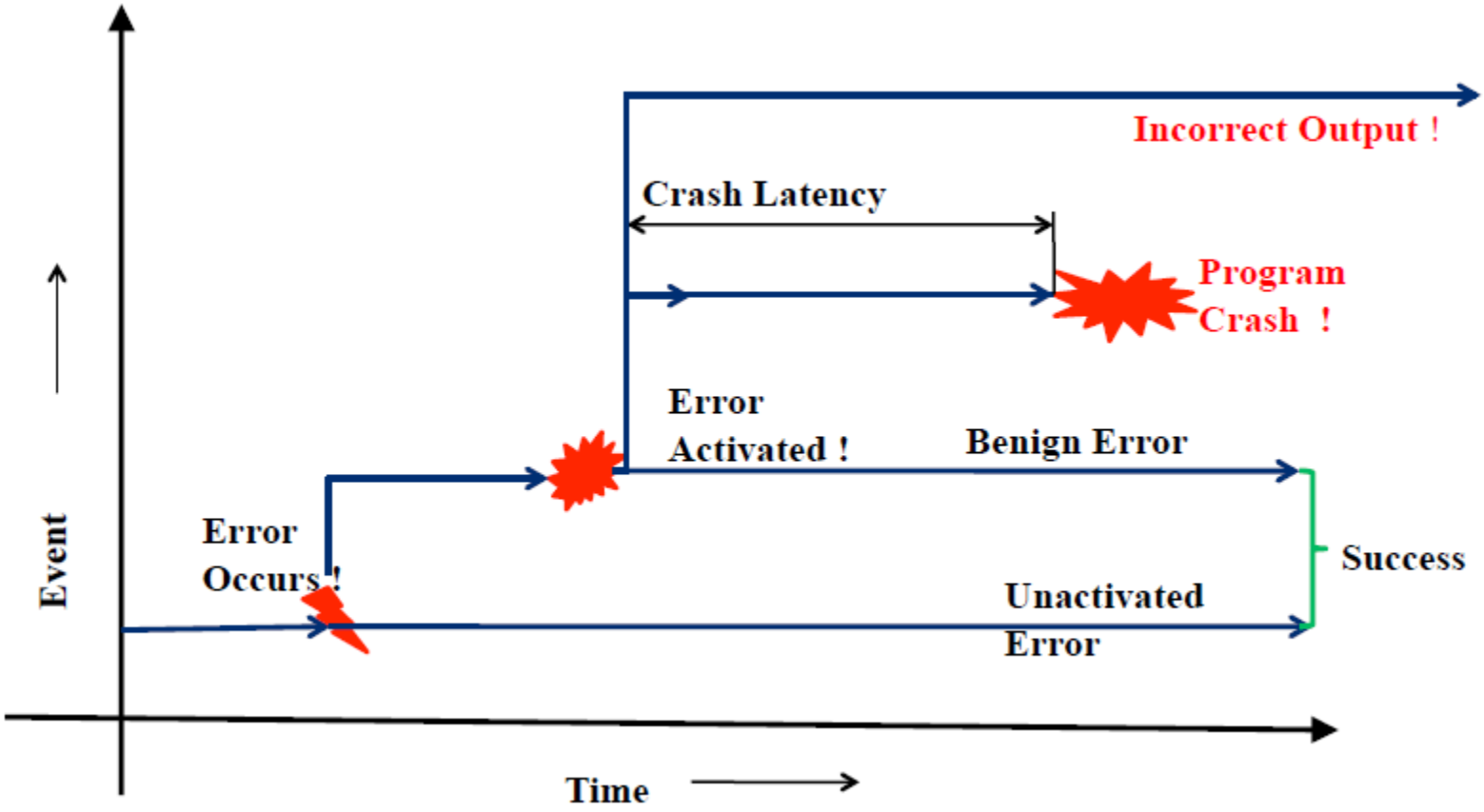
Dependability Impairments



Fault Masking and Benign Errors

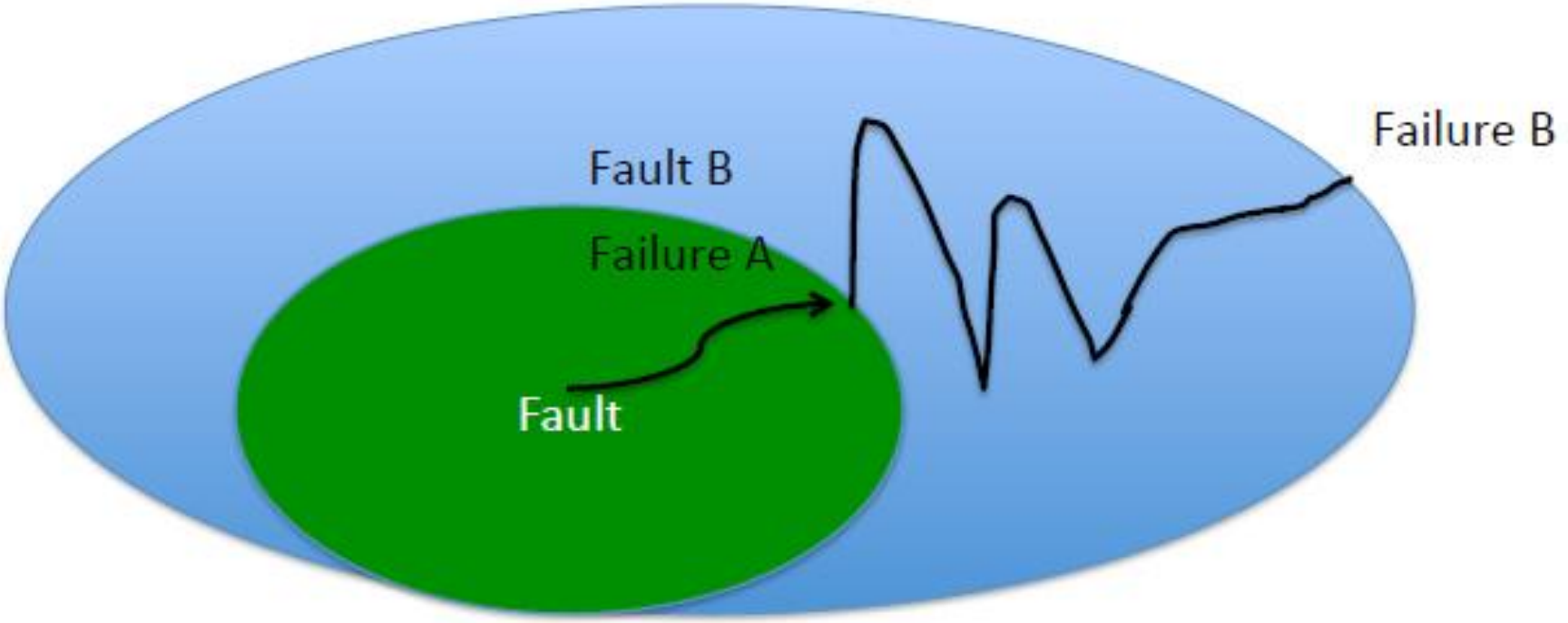
- **Not all faults lead to errors**
  - Faults can be masked because they are not activated (e.g., faults in unread locations)
  - Faults can also be corrected before they lead to errors (e.g., memory scrubbing in ECC)
- **Not all errors lead to failures (Benign errors)**
  - Error may affect inconsequential system state
  - System may have redundancy to correct error

Anatomy of an Error



Inter-connected Systems

- One system's failure (system A) may be another one's fault/error (system B)



Fault Classification

Temporal

- Transient
  - Occur only once at a location
  - E.g., cosmic ray strikes
- Intermittent
  - Periodically recur at a location
  - E.g., timing violations
- Permanent
  - Continuous and stable occurrence at a location
  - E.g., stuck-at-faults

Origin

- Physical faults
  - Occur due to physical phenomena such as EM effects, threshold, effects etc. or due to environmental conditions such as temperature or workload
- Human-made faults
- Programming errors, mis-configuration, human operator errors, etc

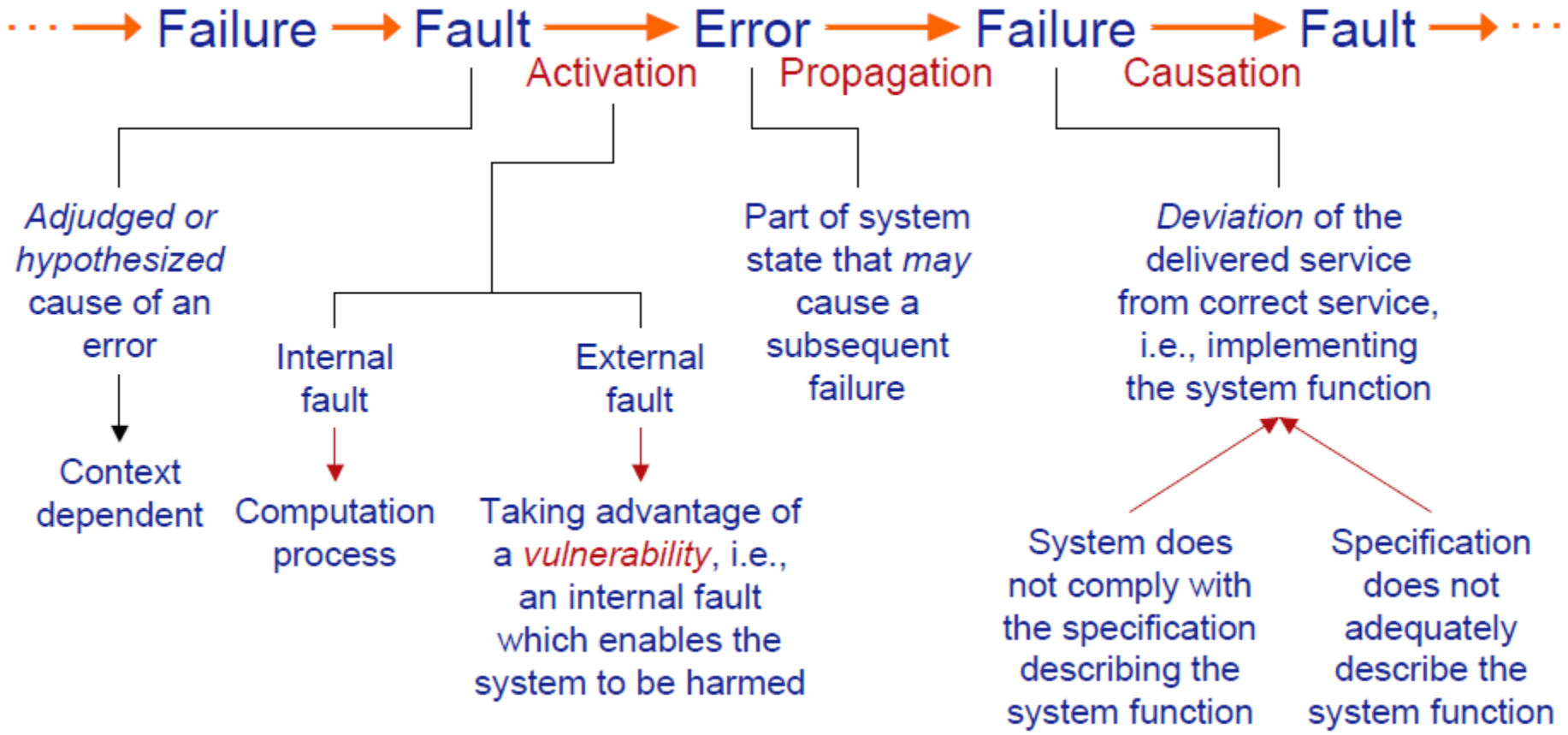
Classification of Failures

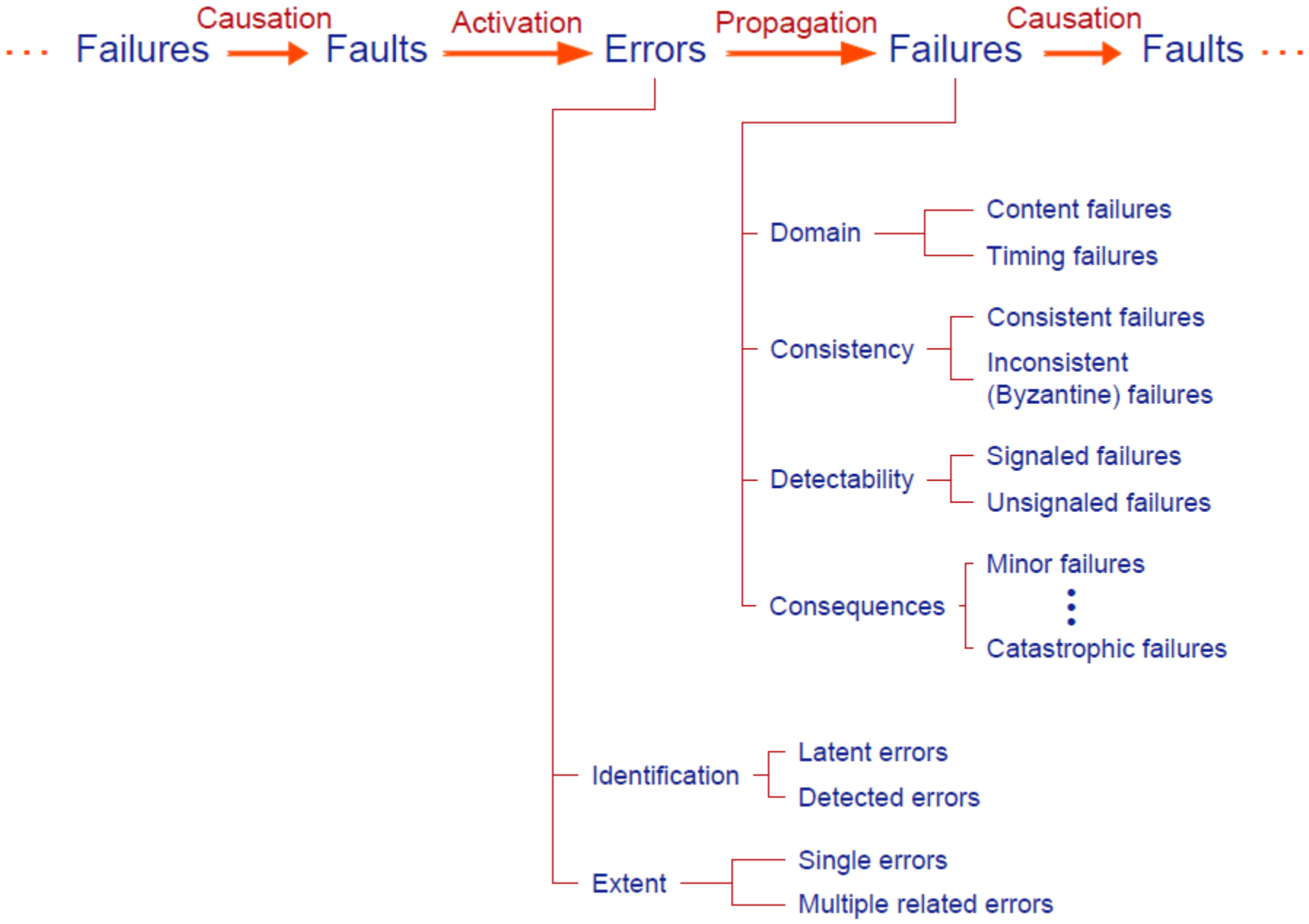
Criteria	Classification 1	Classification 2
Nature	Halt (fail-stop)	Erratic (Babbling)
Detection	Signaled	Un-signaled (fail silent)
Consistency	Consistent	Inconsistent
Severity	Minor	Catastrophic

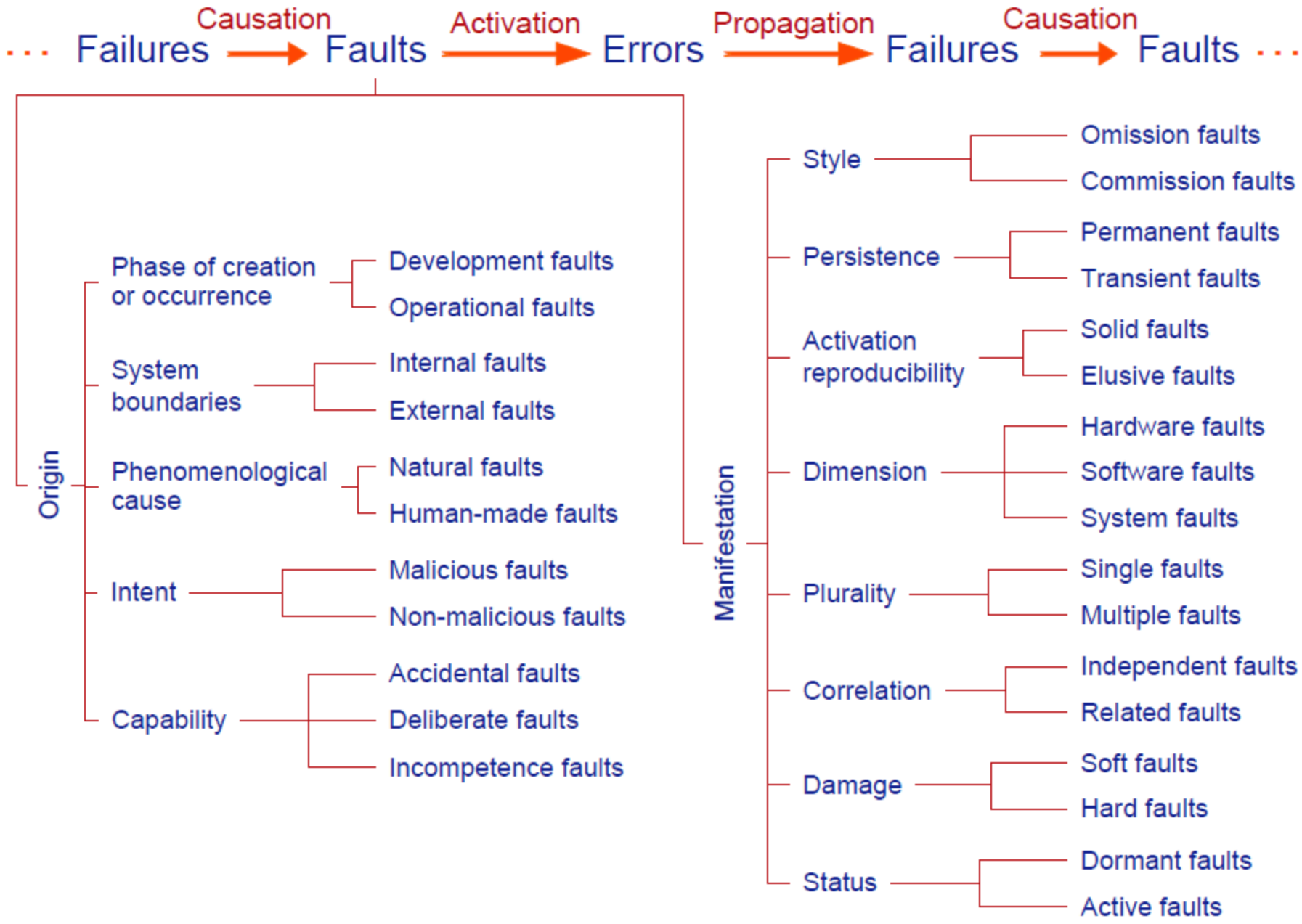
Classification of Errors

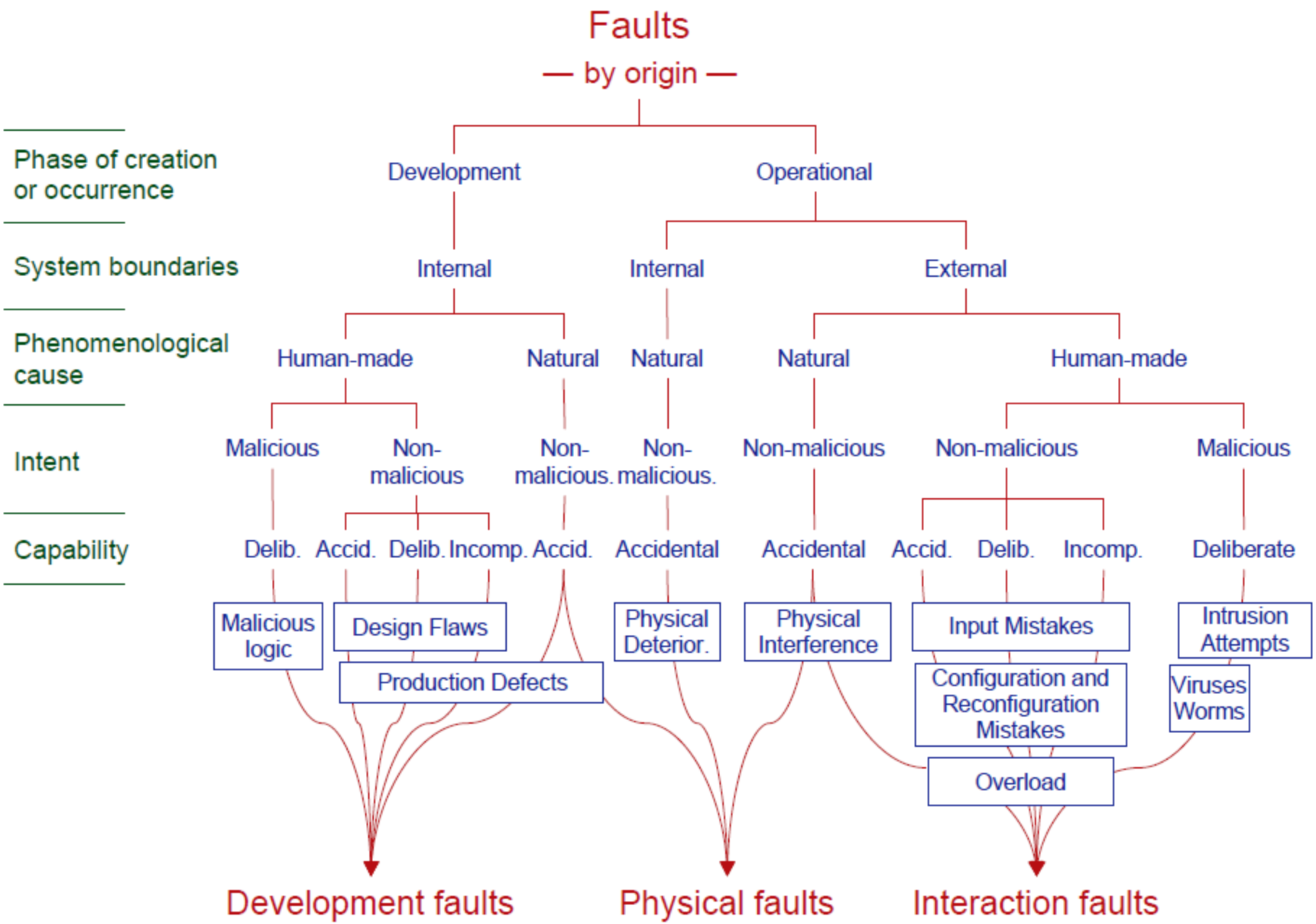
Criteria	Classification 1	Classification 2
Domain	Timing	Content
Detection	Detected	Latent
Consistency	Consistent	Inconsistent
Severity	Minor	Catastrophic

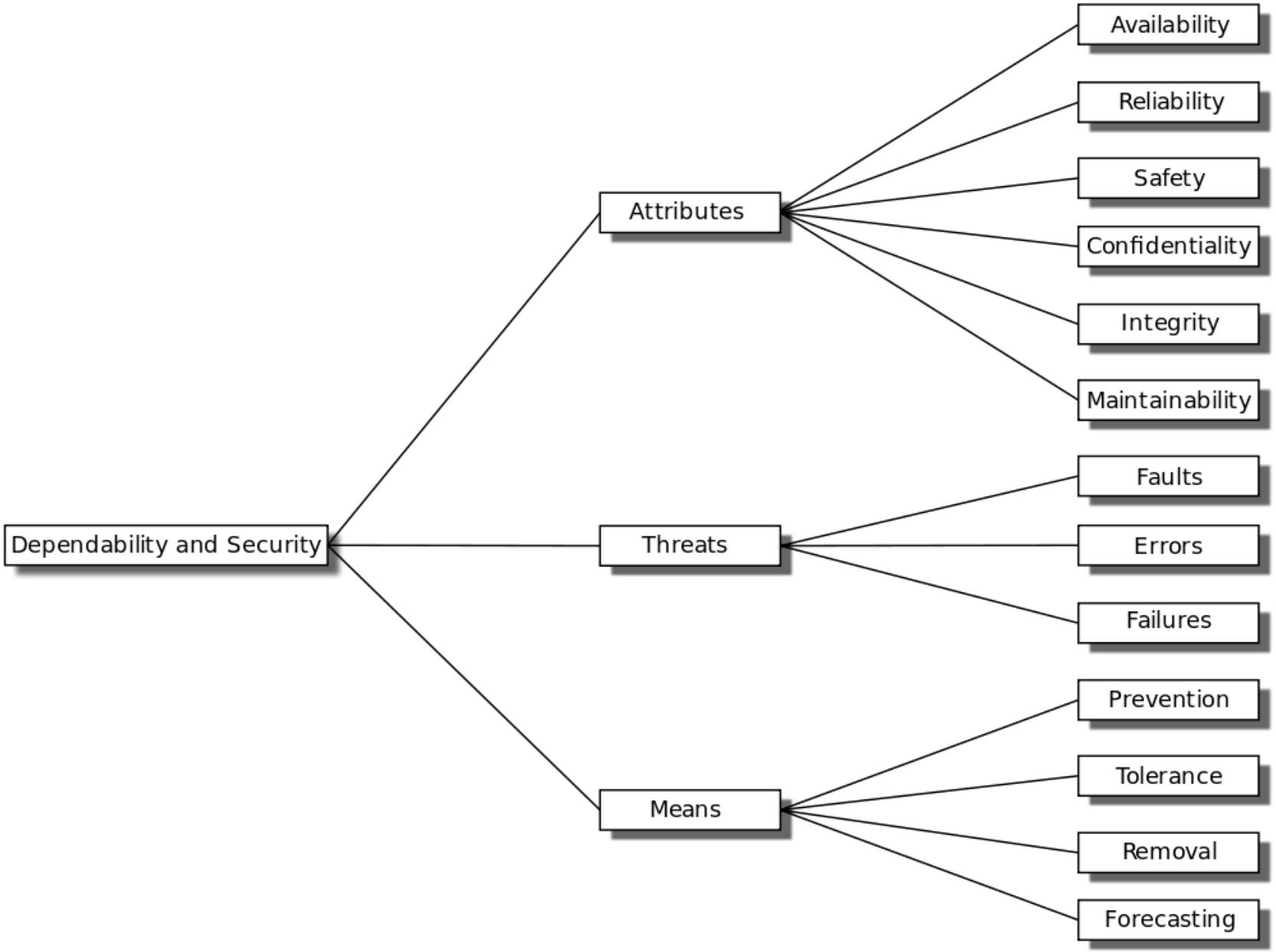
Dependability Threats




Causation
Activation
Propagation
Causation
Failures
Faults
Errors
Failures
Faults
Domain
Content failures
Timing failures
Consistency
Consistent failures
Inconsistent
(Byzantine) failures
Detectability
Signaled failures
Unsignaled failures
Consequences
Minor failures
Catastrophic failures
Identification
Latent errors
Detected errors
Extent
Single errors
Multiple related errors


Causation
Activation
Propagation
Causation
Failures
Faults
Errors
Failures
Faults
Origin
Phase of creation or occurrence
Development faults
Operational faults
System boundaries
Internal faults
External faults
Phenomenological cause
Natural faults
Human-made faults
Intent
Malicious faults
Non-malicious faults
Capability
Accidental faults
Deliberate faults
Incompetence faults
Manifestation
Style
Omission faults
Commission faults
Persistence
Permanent faults
Transient faults
Activation reproducibility
Solid faults
Elusive faults
Dimension
Hardware faults
Software faults
System faults
Plurality
Single faults
Multiple faults
Correlation
Independent faults
Related faults
Damage
Soft faults
Hard faults
Status
Dormant faults
Active faults


Faults
— by origin —
Phase of creation or occurrence
System boundaries
Phenomenological cause
Intent
Capability
Development
Operational
Internal
Internal
External
Human-made
Natural
Natural
Natural
Human-made
Malicious
Non-malicious
Non-malicious.
Non-malicious.
Non-malicious
Non-malicious
Malicious
Delib.
Accid.
Delib.
Incomp.
Accid.
Accidental
Accidental
Accid.
Delib.
Incomp.
Deliberate
Malicious logic
Design Flaws
Production Defects
Physical Deterior.
Physical Interference
Input Mistakes
Configuration and Reconfiguration Mistakes
Overload
Intrusion Attempts
Viruses Worms
Development faults
Physical faults
Interaction faults


Dependability and Security
Attributes
Threats
Means
Availability
Reliability
Safety
Confidentiality
Integrity
Maintainability
Faults
Errors
Failures
Prevention
Tolerance
Removal
Forecasting

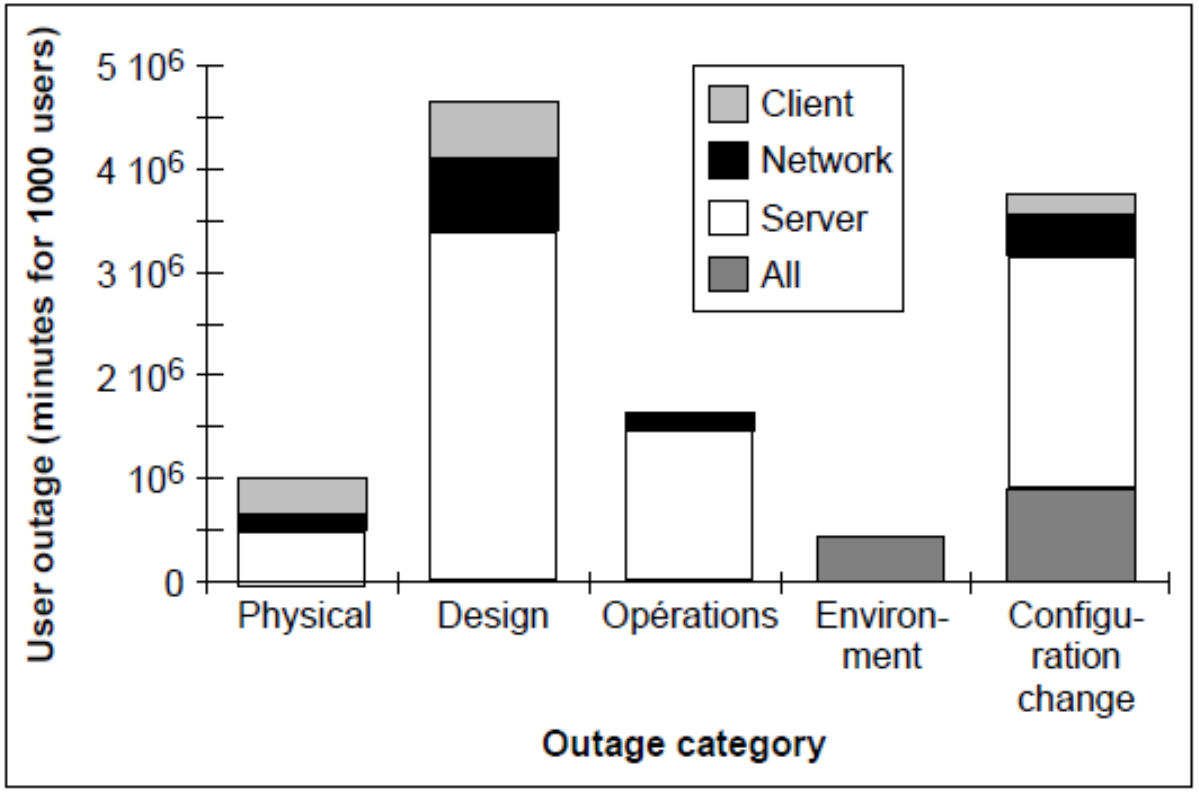
Limits of Dependability

- Limits in term of Fault Classes
  - Physical Faults
  - Design/Development Faults
  - Interaction Faults
- Software Reliability
  - Current Bottleneck of Computing Systems
- Fault Tolerance Effectiveness

Failure Sources & Frequencies for Computing Systems

Non-fault-tolerant systems (Japan, 1383 organizations [Watanabe 86]; USA, 450 companies [FIND/SVP 93])	**Fault-tolerant systems** (Tandem Computers [Gray 90]; Bell Northern Research [Cramp et al. 92])
Mean Time To Failure: 6 to 12 weeks Average outage duration after failure: 1 to 4 hours	Mean Time To Failure: 21 years [Tandem]
Failure sources Hardware 50% Software 25% Communications-Environment 15% Operations - Procedures 10%	Failure sources Software 65% Operations - Procedures 10% Hardware 8% Environment 7%

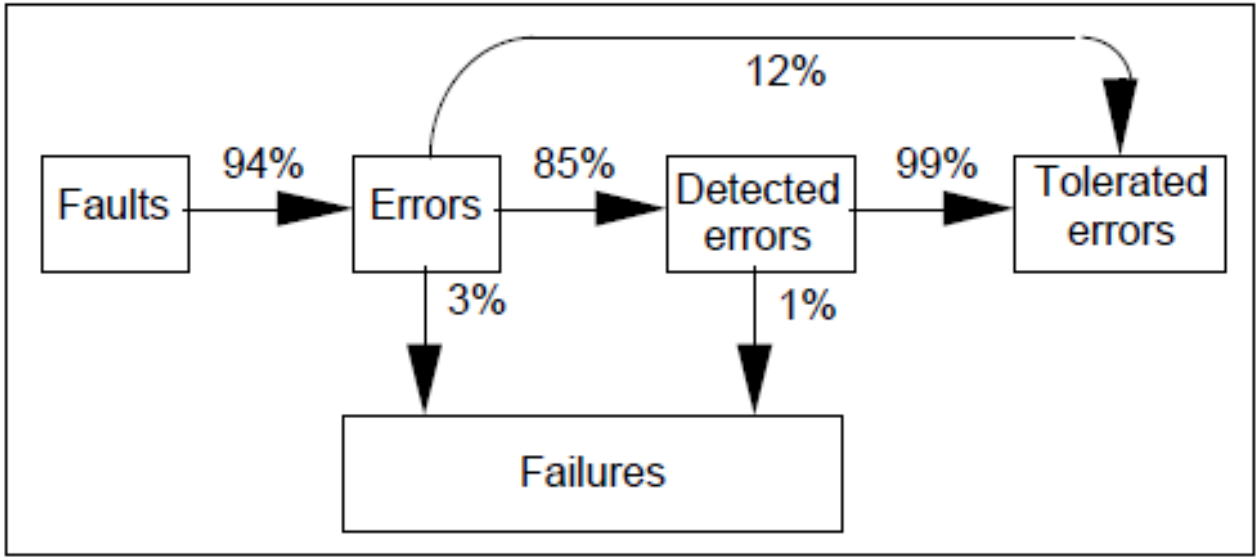
User Outage in Non-Fault-Tolerant Client-Server Networks *[Wood 94]*



Primary Cause of Accident of Domestic Flights in US *[Ruegger 90]*

	Accidents per million takeoffs			
	1970-1978		1979-1986	
Technical defects	1.49	(45%)	0.43	(33%)
Weather	0.82	(25%)	0.33	(26%)
Human error	1.03	(30%)	0.53	(41%)
Total	3.34		1.29	

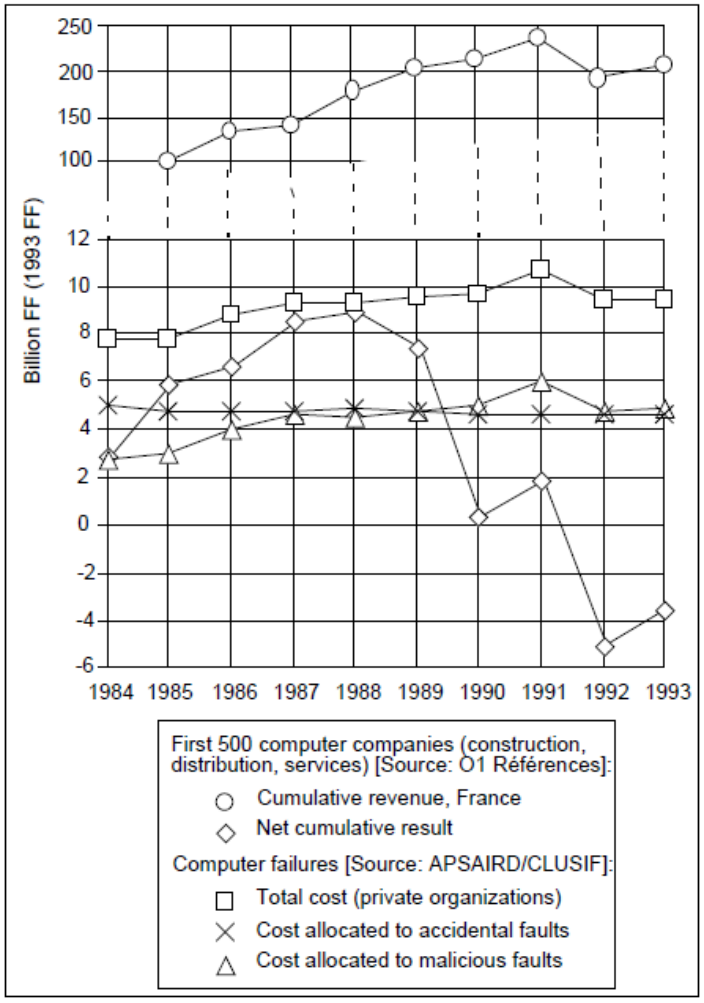
Results of Fault-Injection Experiments in Delta-4 *[Arlat et al.91]*



Challenges of Dependability

- Reducing Cost of Computer Failures
- Extending Fault Tolerance from Products to their Production Processes

Cost of Computer Failures in FRA as compared to the Revenue



FTC incidents in 2025

Incident (2025)	What Happened	Fault Tolerance Lesson
Google Cloud Outage (June 12, 2025)	A configuration change introduced a null-pointer crash loop in Google Service Control, causing failures across many Google Cloud services. The faulty configuration propagated globally within seconds. (The Economic Times)	Global replication without staged rollout created a **common-mode failure**. Better fault tolerance would include feature flags, canary deployments, and regional isolation.
AWS DynamoDB/DNS Outage (October 20, 2025)	A DNS-related failure in the US-East-1 region caused cascading failures across DynamoDB and many AWS services, affecting thousands of applications worldwide. (TechRadar)	Even highly redundant cloud systems can fail if a critical dependency becomes a single point of failure. Multi-region and multi-cloud architectures improve resilience.
Microsoft Azure Front Door Outage (October 29, 2025)	A configuration update prevented Azure Front Door nodes from loading correctly worldwide until Microsoft rolled back the change. (ThousandEyes)	Geographic redundancy cannot help when every region receives the same faulty configuration. Progressive rollouts and automated rollback mechanisms are essential.
Slack Database Outage (February 26, 2025)	A maintenance action overloaded database systems, causing messaging failures for several hours despite healthy network connectivity. (Network World)	Fault tolerance must cover backend databases as well as network infrastructure. Monitoring should detect application-layer failures, not just network health.
Asana Consecutive Outages (February 5–6, 2025)	Two configuration-related incidents caused service disruptions after server logs overwhelmed systems. Afterward, Asana adopted staged configuration rollouts. (ThousandEyes)	Small configuration errors can cascade. Canary deployments and gradual rollouts are effective fault-tolerance practices.
Cloudflare Outage (November 18, 2025)	A latent software bug triggered by a routine configuration change caused widespread degradation affecting numerous Internet services. (CIO)	Hidden software defects can bypass redundant infrastructure. Extensive testing and automatic rollback reduce the impact.
Equinix Singapore Data Center Outage (May 31, 2025)	Maintenance temporarily reduced redundancy, and an equipment fault caused two data halls to go offline. (Data Center Frontier)	Maintenance windows can weaken fault tolerance. Critical infrastructure should preserve sufficient redundancy during planned work.

Assignment HW#01

- Answer the following question in one page (A4), and submit via **myCourseVille** by ***17 Aug 2026***
- Show a **Threat Chain on Machine Learning** system. *Please select one incident and explain it in detail.*
- *Example:* Threat on Server Hardware & Cloud Database
  - **Fault:** A memory chip (RAM) inside a cloud server degrades over time and suffers a hardware defect.
  - **Error:** Data bits stored in that specific RAM location flip from `1` to `0` (Data Corruption).
  - **Failure:** A user tries to download their profile photo, but the website displays a broken image icon and a "500 Internal Server Error" message.

The ML Threat Chain

[Fault]

Root Defect

- *Bad training data*
- *Hidden bias*
- *Poisoned dataset*

[Error]

Internal State

- *Corrupted weights*
- *Out-of-distribution*
- *Overfitting*

[Failure]

Visible Misbehavior

- *Hallucinatory output*
- *Catastrophic system*